

Types of AI Agents

A Comprehensive Guide

Understanding Agent Architectures, Capabilities, and Use Cases

What is an AI Agent?

An AI agent is an autonomous system that perceives its environment, makes decisions, and takes actions to achieve specific goals. Unlike simple LLMs that generate text based on prompts, agents can interact with tools, maintain memory, plan multi-step workflows, and adapt their behavior based on feedback.

Core Characteristics of AI Agents

- **Autonomy:** Can operate independently without constant human guidance
- **Goal-directed behavior:** Works towards specific objectives
- **Environmental perception:** Gathers information from its context
- **Action capability:** Can execute operations and use tools
- **Adaptability:** Learns and adjusts strategies based on outcomes

LLM vs LLM with Tools vs AI Agent

Understanding the progression from basic LLMs to full agents is crucial:

Component	LLM Only	LLM + Tools	AI Agent
Input/Output	Text in, text out	Text + function calls	Multi-modal I/O
Memory	Context window only	Stateless per call	Persistent memory
Planning	Single response	Tool selection	Multi-step reasoning
Autonomy	None	Reactive	Proactive

Learning	Fixed (pretrained)	Fixed (pretrained)	Can adapt behavior
Example	ChatGPT base	Function calling API	AutoGPT, ReAct agents

Key Insight

An LLM with tools is not necessarily an agent. The critical difference is autonomy and goal-directed behavior. An agent decides WHEN and HOW to use tools based on its goals, while a simple LLM + tools setup requires explicit instructions for each tool use.

1. Reactive Agents (ReAct)

Reactive agents follow a simple loop: Reason about the current situation, then Act. This is the most common and straightforward agent architecture.

Architecture

Flow: User Query → Think → Act → Observe → Think → Act → ... → Final Answer

Key Components

- Reasoning Engine: LLM that generates thoughts about what to do next
- Action Space: Set of available tools (search, calculator, API calls, etc.)
- Observation: Feedback from tool execution
- Memory: Conversation history stored in context

Decision Flow

Step	Process	Example
1. Thought	LLM reasons about next action	"I need to search for current weather"
2. Action	Execute tool call	search("weather in San Francisco")
3. Observation	Receive tool output	"72°F, sunny, light breeze"
4. Thought	Evaluate result	"I have the weather info, can answer now"

5. Action	Provide final answer	"It's 72°F and sunny in San Francisco"
-----------	----------------------	--

Characteristics

- Pros:
 - Simple to implement and understand
 - Works well for straightforward tasks
 - Low latency for single-step queries
 - Easy to debug (clear reasoning trace)
- Cons:
 - No long-term planning
 - Can get stuck in loops
 - Limited memory (context window only)
 - Struggles with complex multi-step tasks

Best Use Cases

- Web browsing and information retrieval
- Simple API orchestration
- Question answering with tool access
- Code execution and debugging

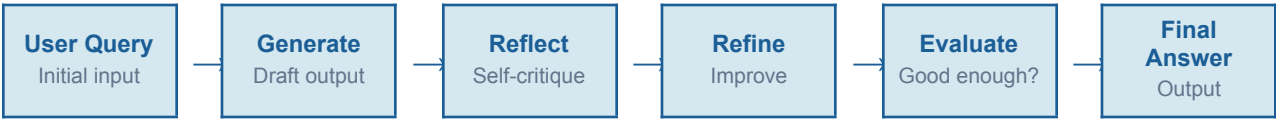
Real-World Example

Customer Support Bot: User asks "What's the status of order #12345?" → Agent thinks: "Need to check order database" → Calls `get_order_status(12345)` → Receives: "Shipped, arriving tomorrow" → Responds: "Your order has been shipped and will arrive tomorrow."

2. Reflection Agents

Reflection agents extend reactive agents by adding a self-evaluation loop. After producing an output, the agent critiques its own work and iterates until the result meets quality criteria. This introspective capability significantly improves output quality for complex tasks.

Architecture



Key Components

Generation Engine

The primary LLM that produces initial drafts or solutions based on the user's query.

Reflection Module

A second pass (same or different LLM) that evaluates the output against defined quality criteria such as correctness, completeness, and clarity.

Refinement Loop

Iterates generation and reflection until quality thresholds are met or a maximum iteration count is reached.

Decision Flow

Step	Process	Example
1. Generate	Produce initial response	Write a code function
2. Reflect	Identify issues in output	"Missing error handling, edge cases"
3. Refine	Improve based on critique	Add try/catch, null checks
4. Re-Reflect	Evaluate improved output	"Looks good, tests would help"
5. Finalize	Output meets quality bar	Deliver polished function

Characteristics

Pros

- Significantly higher output quality through self-improvement
- Catches errors and inconsistencies before delivery
- Works well for code generation, writing, and analysis
- Reduces hallucinations via self-checking

Cons

- Higher latency due to multiple LLM calls
- Increased token costs per request
- Risk of over-refining (diminishing returns)
- Requires well-designed critique prompts

Best Use Cases

- Code generation and debugging
- Long-form content writing and editing
- Mathematical reasoning and proof checking
- Complex data analysis with quality requirements

Real-World Example

Code Review Agent: Developer submits a function → Agent generates implementation → Reflects: "No input validation, $O(n^2)$ complexity" → Refines with validation and optimized algorithm → Re-reflects: "Clean, efficient, well-documented" → Returns polished code with tests.

3. Plan-and-Execute Agents

Plan-and-Execute agents separate the planning phase from the execution phase. A planner LLM first decomposes a complex task into a sequence of steps, and then an executor agent carries out each step. This separation allows for more deliberate, structured problem-solving.

Architecture



Key Components

Planner LLM

A high-capacity model (often a more powerful LLM) that receives the user goal and produces a structured plan — a numbered list of sub-tasks with dependencies.

Executor Agent

A reactive sub-agent that processes each plan step, uses available tools, and reports results back to the planner.

State Tracker

Maintains the current plan, completed steps, results, and context so the planner can replan dynamically if a step fails or yields unexpected results.

Decision Flow

Step	Process	Example
1. Decompose	Break goal into steps	"Research → Analyze → Write → Review"
2. Assign	Map steps to tools/agents	Search tool, Python executor, LLM writer
3. Execute	Run steps sequentially	Search for recent AI papers
4. Evaluate	Check step results	"Found 10 papers, extracting key insights"
5. Replan	Adjust remaining steps	Add summarization step if too much data
6. Deliver	Compile final output	Structured research report delivered

Characteristics

Pros

- Handles complex, multi-step tasks that require coordination
- Explicit plan makes reasoning transparent and debuggable
- Can recover from failures by replanning
- Better long-horizon task completion

Cons

- Higher latency: planning adds an extra LLM call
- Plan quality depends heavily on the planner LLM
- Rigid plans can fail on dynamic, unpredictable tasks
- Complex to implement with robust error handling

Best Use Cases

- Research and report generation
- Software project scaffolding and implementation
- Multi-step data pipelines and ETL workflows
- Travel and event planning assistants

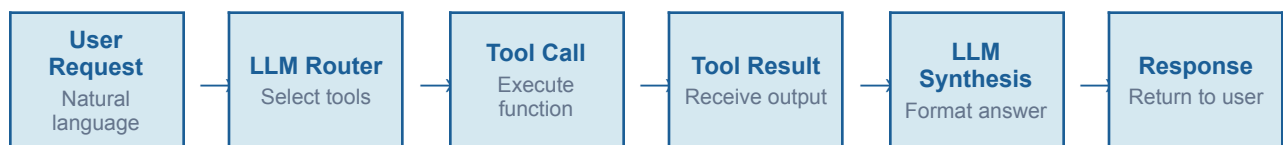
Real-World Example

Research Assistant: User asks "Write a competitive analysis of top 5 CRM tools" → Planner creates: [1. List competitors, 2. Research each, 3. Compare features, 4. Analyze pricing, 5. Write report] → Executor runs web searches per step → Planner reviews intermediate results → Delivers structured competitive analysis document.

4. Tool-Using Agents (Function Calling)

Tool-using agents leverage LLM function-calling capabilities to interact with external APIs, databases, code interpreters, and services. The LLM decides which tools to invoke based on the user request and constructs the appropriate parameters automatically.

Architecture



Key Components

Tool Registry

A catalog of available functions with schemas describing their parameters, return types, and descriptions. The LLM uses these schemas to decide when and how to call each tool.

Function Dispatcher

Middleware that receives structured tool-call requests from the LLM, executes the corresponding function in the real environment, and returns results.

Result Integrator

The LLM processes tool outputs and integrates them into a coherent, natural-language response for the user.

Common Tool Categories

Tool Type	Examples	Use Case
Search & Retrieval	Web search, Vector DB, RAG	Fetch current or domain-specific info
Code Execution	Python REPL, Bash, SQL	Run calculations, data analysis
External APIs	Weather, Maps, Calendar	Access third-party services
File Operations	Read, Write, Parse files	Document processing and storage
Communication	Email, Slack, SMS	Send messages and notifications
Data Services	CRM, ERP, Databases	Read/write business data

Characteristics

Pros

- Highly flexible: easy to extend by adding new tools
- Grounds LLM responses in real, current data
- Native support in OpenAI, Anthropic, and Google APIs
- Low latency for single-tool queries

Cons

- LLM may hallucinate tool parameters if schemas are unclear
- Security risks if tools have write/execute access
- Multiple sequential tool calls can add latency
- Tool selection errors can cascade into wrong results

Best Use Cases

- Personal assistants with calendar and email integration
- Data analytics bots with database access
- Developer tools with code execution capabilities
- Customer-facing bots with CRM and ticketing integration

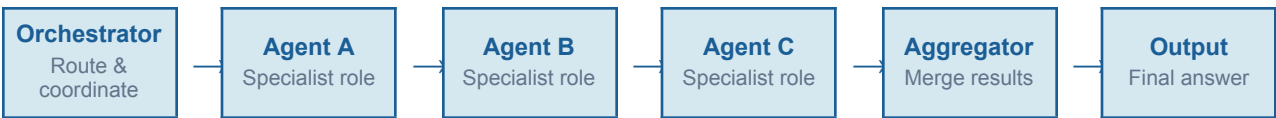
Real-World Example

Financial Assistant: User asks "What is Apple's current P/E ratio vs last year?" → Agent calls `get_stock_data(ticker="AAPL", metrics=["PE_ratio"])` → Calls `get_historical_data(ticker="AAPL", period="1Y")` → Receives structured JSON → Synthesizes: "Apple's P/E is currently 28.4, down from 31.2 a year ago, suggesting improved value."

5. Multi-Agent Systems

Multi-Agent Systems (MAS) consist of multiple specialized AI agents that collaborate, communicate, and coordinate to solve problems beyond the capability of a single agent. Each agent has distinct expertise, tools, and responsibilities within a shared workflow.

Architecture



Key Components

Orchestrator Agent

A central coordinator that receives the user's request, breaks it into sub-tasks, routes each sub-task to the appropriate specialist agent, and aggregates results into a final response.

Specialist Agents

Purpose-built agents with focused capabilities — for example, a `ResearchAgent`, a `CodeAgent`, a `WritingAgent`, and a `FactCheckAgent` — each with their own tool sets and system prompts.

Communication Protocol

A shared messaging format and context-passing mechanism that allows agents to share intermediate results, request assistance from peers, and avoid duplicating work.

Collaboration Patterns

Pattern	Description	When to Use
Sequential Pipeline	Agents hand off results in order	Structured, dependent workflows

Parallel Execution	Agents run simultaneously	Independent subtasks
Hierarchical	Manager → worker agents	Complex delegation trees
Peer-to-Peer	Agents consult each other	Dynamic problem solving
Debate / Critique	Agents argue for best answer	High-accuracy requirements

Characteristics

Pros

- Enables specialization: each agent excels at its domain
- Scales to very complex, long-horizon tasks
- Parallel execution reduces wall-clock latency
- Modular: agents can be added or replaced independently

Cons

- High orchestration complexity and debugging difficulty
- High cost: multiple LLM calls per request
- Risk of inter-agent miscommunication and error propagation
- Context management across agents is challenging

Best Use Cases

- End-to-end software development pipelines
- Enterprise knowledge management and Q&A systems
- Complex scientific research automation
- Multi-department business process automation

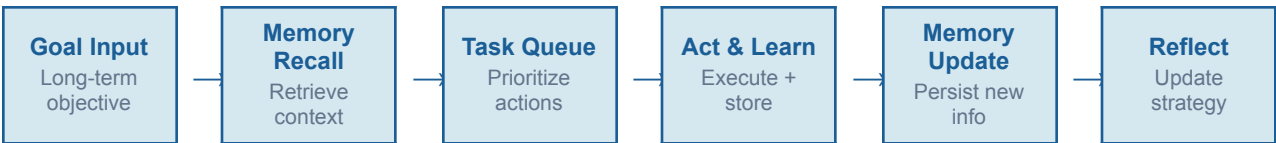
Real-World Example

Software Engineering MAS: User requests a new feature → OrchestratorAgent routes to: ArchitectAgent (designs schema), CodeAgent (writes implementation), TestAgent (generates unit tests), ReviewAgent (checks code quality) → Each runs in parallel → OrchestratorAgent merges results into a complete pull request ready for human review.

6. Autonomous Agents with Memory

Autonomous agents with memory go beyond single-session interactions by maintaining persistent knowledge about users, tasks, and the world across sessions. They proactively pursue long-term goals, manage their own task queues, and leverage episodic, semantic, and procedural memory to become more effective over time.

Architecture



Memory Types

Memory Type	Description	Example Storage
Working Memory	Current context and active task state	LLM context window
Episodic Memory	Past interactions and experiences	Vector database (e.g., Pinecone)
Semantic Memory	Facts, knowledge, user preferences	Knowledge graph or SQL DB
Procedural Memory	How to perform recurring tasks	Fine-tuned weights or few-shot prompts

Key Components

Memory Manager

Handles reading from and writing to different memory stores. Decides what information is worth persisting and what can be discarded after a session.

Task Queue / Scheduler

Maintains a prioritized list of pending tasks, enabling the agent to proactively work toward long-term goals even without a user prompt.

Reflection Engine

Periodically reviews past actions and outcomes to update strategies, correct misconceptions, and improve future performance — similar to a human reflecting on lessons learned.

Characteristics

Pros

- Personalization improves significantly over time
- Can pursue long-term goals across days or weeks
- Reduces repetitive information gathering across sessions
- Proactive behavior — acts without waiting for prompts

Cons

- Memory storage and retrieval adds infrastructure complexity
- Privacy and data security concerns with persistent user data
- Memory staleness: outdated info can cause incorrect behavior
- Very high cost at scale with continuous background processing

Best Use Cases

- Personal AI assistants that learn user preferences
- Long-running research or project management agents
- AI companions with persistent relationship context
- Business process agents managing ongoing workflows

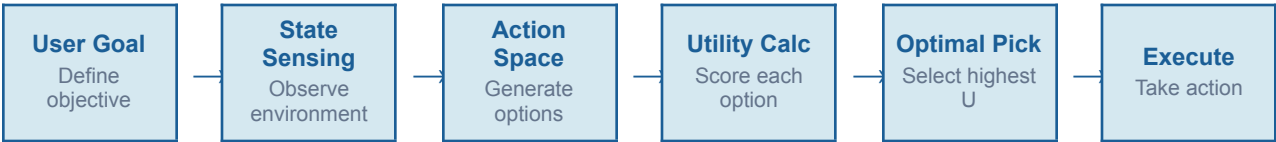
Real-World Example

Personal AI Chief of Staff: Over weeks, the agent learns your meeting preferences, key stakeholders, communication style, and recurring tasks. It proactively drafts Monday briefings from your inbox, schedules follow-ups it remembers you forgot, and refines its behavior each time you give feedback — becoming measurably more useful over time.

7. Utility-Based Agents

Utility-based agents make decisions by evaluating multiple possible actions against a utility function — a formal measure of how desirable each outcome is. Rather than following fixed rules, these agents optimize for the best expected outcome given uncertainty and trade-offs.

Architecture



Key Components

State Estimator

Builds a model of the current world state by gathering information from the environment, user history, and available tools.

Utility Function

A mathematical or heuristic scoring function that assigns a value to each possible action-outcome pair. Well-designed utility functions encode the user's priorities, constraints, and risk tolerance.

Decision Engine

Evaluates all candidate actions against the utility function (considering uncertainty) and selects the action with the highest expected utility.

Utility Trade-off Examples

Decision	Options Evaluated	Utility Factors
Route planning	3 travel routes	Speed, cost, reliability, safety
Email priority	10 unread emails	Urgency, sender importance, topic relevance
Investment	5 portfolio options	Expected return, risk, liquidity, timeline
Resource allocation	4 server configs	Cost, latency, availability, scalability

Characteristics

Pros

- Principled decision-making under uncertainty
- Naturally handles trade-offs between competing objectives
- Transparent: utility scores explain why choices were made

- Adaptable: utility function can be tuned per user or context

Cons

- Designing accurate utility functions is difficult in practice
- Computationally expensive for large action spaces
- Utility functions can encode biases or incorrect priorities
- May underperform reactive agents on simple, well-defined tasks

Best Use Cases

- Autonomous trading and financial decision systems
- Resource scheduling and operations optimization
- Recommendation engines with multi-criteria ranking
- Autonomous robotics and navigation systems

Real-World Example

Smart Travel Planner: User says "Get me to the conference by 9am, minimize cost, avoid traffic" → Agent enumerates 8 route/mode combinations → Computes utility score for each across cost, time, reliability → Selects optimal option → Books ride and sets departure alert, explaining: "Uber at 7:15am scores highest: \$18, 95% on-time probability."

Comparative Summary

Choosing the right agent architecture depends on your specific use case, complexity requirements, and available resources.

Agent Type	Complexity	Latency	Cost	Autonomy Level
Reactive (ReAct)	Low	Low	Low	Low
Reflection	Medium	Medium	Medium	Low-Medium
Plan-Execute	Medium-High	Medium	Medium	Medium
Tool-Using	Low-Medium	Low-Medium	Low-Medium	Low
Multi-Agent	High	High	High	Medium-High

Autonomous + Memory	Very High	Variable	High	Very High
Utility-Based	Medium-High	Medium-High	Medium	Medium
